



Projet interdisciplinaire d'exploration de données

Higgs Boson Machine Learning Challenge

Plottier Louis / Abkress Lawane-Adjid / Manondzo De Mitsoko Wath Ahira

11 mai 2026

Table des matières

1	Introduction au projet : objectif et métrique	5
2	Analyse exploratoire	5
2.1	Taille des dataframes	5
2.2	Analyse des colonnes (PRI, DER, etc.)	5
2.3	Répartition des classes	6
2.4	Interprétation physique des données manquantes	6
2.5	Analyse de la variable <i>PRI_jet_num</i>	6
2.5.1	Séparation des données selon le nombre de jets	7
2.6	Analyse du pouvoir discriminant des variables	7
2.6.1	Résultats observés	7
2.7	Analyse des corrélations	8
2.8	Croisement entre corrélation et pouvoir discriminant	9
2.9	Analyse de la colonne <i>Weight</i> et rôle des poids statistiques	10
2.10	Interprétation physique	11
2.11	Impact sur l'évaluation des modèles	11
2.12	Justification de l'AMS	11
2.13	Conclusion de l'exploration des données	12
3	Feature Engineering	12
3.1	Importance du feature engineering	12
3.2	Explication des nouvelles colonnes	12
3.3	Data Preprocessing : nettoyage et séparation des données	14
4	Modèle, Seuil et Optimisation	15
4.1	Explication des modèles	15
4.1.1	Choix du type de modèle	15
4.1.2	XGBoost	15
4.1.3	CatBoost	16
4.1.4	LightGBM	17
4.2	Optimisation du seuil	17
4.2.1	Explication du seuil	17

4.2.2	XGBoost	18
4.2.3	CatBoost	19
4.2.4	LightGBM	19
4.3	Optimisation des hyperparamètres des modèles	20
4.3.1	XGBoost	20
4.3.2	CatBoost	21
4.3.3	LightGBM	22
4.3.4	Conclusion sur l’optimisation des hyperparamètres des modèles	23
4.4	Scores	23
4.4.1	Explication du score Kaggle	23
4.4.2	XGBoost	23
4.4.3	CatBoost	24
4.4.4	LightGBM	24
4.4.5	Importances des colonnes pour le XGBoost	24
4.4.6	Conclusion sur les scores	25
4.5	Autres approches	25
4.5.1	Réseau de neurones	25
4.5.2	Stacking de modèles	26
4.5.3	Décomposition suivant PRI_jet_num	26
5	Autoencodeur	26
5.1	Ajout de colonnes supplémentaire	26
5.2	Utilité de l’Autoencodeur	28
5.3	Problème des valeurs manquantes	29
5.3.1	La nature physique des -999	29
5.4	La stratégie d’imputation adoptée	29
5.5	Recherche de la dimension optimale du goulot d’étranglement	30
5.5.1	Principe de la méthode du coude	30
5.5.2	Résultats – Graphiques de sélection	30
5.5.3	Convergence de l’entraînement final	31
5.5.4	Analyse de l’espace latent — PCA 2D	31
6	Conclusion	32

1 Introduction au projet : objectif et métrique

L'objectif de ce projet est de classifier des événements de collision de particules pour isoler le signal du boson de Higgs. Pour réussir ce challenge, il est crucial de comprendre les notions suivantes :

- **Le Signal (s)** : Événements issus de la désintégration du boson de Higgs en deux leptons tau. C'est ce que nous cherchons à détecter.
- **Le Bruit de fond (b)** : Événements produits par d'autres processus physiques (comme la désintégration de bosons Z) qui imitent la signature du Higgs.
- **L'AMS (Approximate Median Significance)** : C'est la métrique d'évaluation officielle du challenge. Contrairement à l'accuracy, l'AMS mesure la force statistique de la découverte. Elle pénalise fortement les faux positifs car, en physique, annoncer une fausse découverte est plus grave que de rater un événement rare.

$$AMS = \sqrt{2 \left((s + b + b_r) \log \left(1 + \frac{s}{b + b_r} \right) - s \right)} \quad (1)$$

Où s et b sont respectivement les taux de vrais positifs et de faux positifs (pondérés), et $b_r = 10$ est un terme de régularisation utilisé pour réduire la variance de la métrique pour les petits échantillons de bruit. Pour mieux comprendre l'AMS, nous pourrions utiliser l'approximation suivante :

$$AMS \approx \frac{s}{\sqrt{b + 10}} \quad (2)$$

Nous reviendrons ultérieurement sur le calcul de l'AMS pour mieux appréhender comment cette métrique fonctionne.

2 Analyse exploratoire

2.1 Taille des dataframes

Les deux dataframes sont de la taille suivante :

- **Train set** : 250 000 lignes et 33 colonnes.
- **Test set** : 550 000 lignes et 31 colonnes.

Le jeu de test est plus de deux fois plus grand que celui d'entraînement. Avoir une telle quantité de données de test permet d'assurer une évaluation très robuste de la performance du modèle (mesurée par la métrique AMS), limitant ainsi les risques d'overfitting sur le classement public de la compétition.

Par ailleurs, la différence du nombre de colonnes (33 contre 31) s'explique par la présence, dans le fichier d'entraînement, de la colonne cible (*Label*) et de la colonne poids (*Weight*), indispensables pour l'apprentissage mais absentes du fichier de test final.

2.2 Analyse des colonnes (PRI, DER, etc.)

Une inspection des colonnes du jeu d'entraînement permet de regrouper les variables en trois catégories distinctes, selon leur origine et leur rôle dans l'analyse :

Variables Primaires (PRI) : Ce sont des mesures directes effectuées par les détecteurs du CERN lors des collisions (ex : *momentum* transverse, angles η et ϕ). Elles représentent l'état brut de l'événement.

Variables Dérivées (DER) : Ces colonnes sont calculées à partir des variables primaires par les physiciens. Elles sont souvent plus riches en information car elles traduisent des quantités physiques complexes (comme la masse invariante) qui aident à discriminer plus facilement le signal du bruit.

Métadonnées et Cibles :

- **EventId** : L'identifiant unique de chaque collision.
- **Weight** : Le poids statistique permettant de corriger les simulations pour qu'elles correspondent aux fréquences d'observation réelles lors des collisions.
- **Label** : La variable cible à prédire (s pour signal, b pour bruit).

2.3 Répartition des classes

L'analyse de la variable cible révèle un déséquilibre entre les deux populations d'événements :

- **Signal (s)** : 85 667 événements (34,27 %).
- **Bruit de fond (b)** : 164 333 événements (65,73 %).

Ce ratio de 1 pour 2 confirme que le signal est minoritaire, ce qui justifie l'utilisation de la métrique AMS plutôt qu'une simple mesure de précision (*accuracy*) pour évaluer la performance du modèle. Cette première analyse permet déjà de comprendre que la détection du boson de Higgs correspond à la recherche d'événements relativement rares parmi un grand nombre d'événements de fond.

2.4 Interprétation physique des données manquantes

L'analyse du jeu de données révèle une présence massive de la valeur -999 . Il est fondamental de comprendre que ces entrées ne correspondent pas à des erreurs de mesure, mais à une absence intrinsèque de données liée à la topologie des collisions. En physique des particules, certaines variables (comme la masse invariante de deux jets ou les caractéristiques d'un second jet) ne peuvent être calculées que si un nombre suffisant de jets est produit lors de la collision. Ainsi, pour un événement ne générant aucun jet ou un seul jet, les variables relatives aux jets secondaires n'ont tout simplement pas de sens physique et ne peuvent pas exister. Ces données manquantes sont donc « structurelles » et non aléatoires : elles encodent directement la géométrie de l'événement physique, une information que les modèles de *Boosting* parviennent à exploiter efficacement.

2.5 Analyse de la variable *PRI_jet_num*

La variable *PRI_jet_num* indique le nombre de jets reconstruits dans chaque événement. Contrairement aux autres variables majoritairement continues, cette variable est discrète et joue un rôle central dans l'organisation physique du dataset. L'analyse de sa distribution montre que les événements ne possèdent pas tous la même topologie : certains événements ne contiennent aucun jet, et d'autres possèdent un ou plusieurs jets.

Cette variable est particulièrement importante car elle conditionne l'existence même de plusieurs autres variables. Ainsi, analyser l'ensemble des données sans tenir compte du nombre de jets reviendrait à mélanger des événements de nature physique différente

2.5.1 Séparation des données selon le nombre de jets

Afin de tenir compte de cette structure physique, les données ont été séparées selon la variable *PRI_jet_num*. Quatre sous-ensembles ont donc été créés : événements avec 0, 1, 2 et 3 jets.

La justification physique est la suivante : certaines variables ne sont définies que dans certains contextes physiques. Par exemple, les variables décrivant le second jet ne sont pertinentes qu'à partir de 2 jets ; certaines masses reconstruites nécessitent plusieurs jets pour être calculées. Séparer les événements selon le nombre de jets permet donc de travailler sur des groupes physiquement plus homogènes.

Il y a également une justification statistique. Cette séparation permet également de réduire artificiellement le nombre de valeurs -999, d'éviter que certaines variables soient dominées par les données manquantes, et aussi de comparer signal et bruit dans des contextes cohérents. Ainsi, la séparation par jets constitue une étape importante à la fois du point de vue physique et statistique.

2.6 Analyse du pouvoir discriminant des variables

L'objectif suivant est d'identifier quelles variables permettent réellement de distinguer signal et bruit. Pour cela, une AUC (Area Under Curve) univariée a été calculée pour chaque variable. L'AUC mesure la capacité d'une variable prise seule à séparer les deux classes. Une AUC proche de 0,5 indique une séparation faible, tandis qu'une AUC proche de 1 montre une forte capacité discriminante.

Certaines variables peuvent cependant être discriminantes dans le sens inverse, c'est-à-dire prendre des valeurs plus faibles pour le signal que pour le bruit. Afin de mesurer uniquement la force de séparation indépendamment du sens, l'AUC corrigée suivante a été utilisée : $AUC_{corrigée} = \max(AUC, 1 - AUC)$. Cette correction permet de comparer directement le pouvoir discriminant des différentes variables.

2.6.1 Résultats observés

Les résultats montrent que les variables importantes dépendent fortement du nombre de jets.

Pour les événements sans jet, les variables les plus discriminantes sont notamment : *DER_mass_transverse_met_lep*, *DER_pt_ratio_lep_tau* et *PRI_tau_pt*.

Pour les événements avec plusieurs jets, les variables liées à la structure des jets deviennent beaucoup plus importantes : *DER_deltaeta_jet_jet*, *DER_mass_jet_jet* et *PRI_prodetta_jet_jet*.

Cette observation confirme l'intérêt de la séparation par nombre de jets : les mécanismes de séparation entre signal et bruit ne sont pas identiques selon la topologie physique de l'événement.

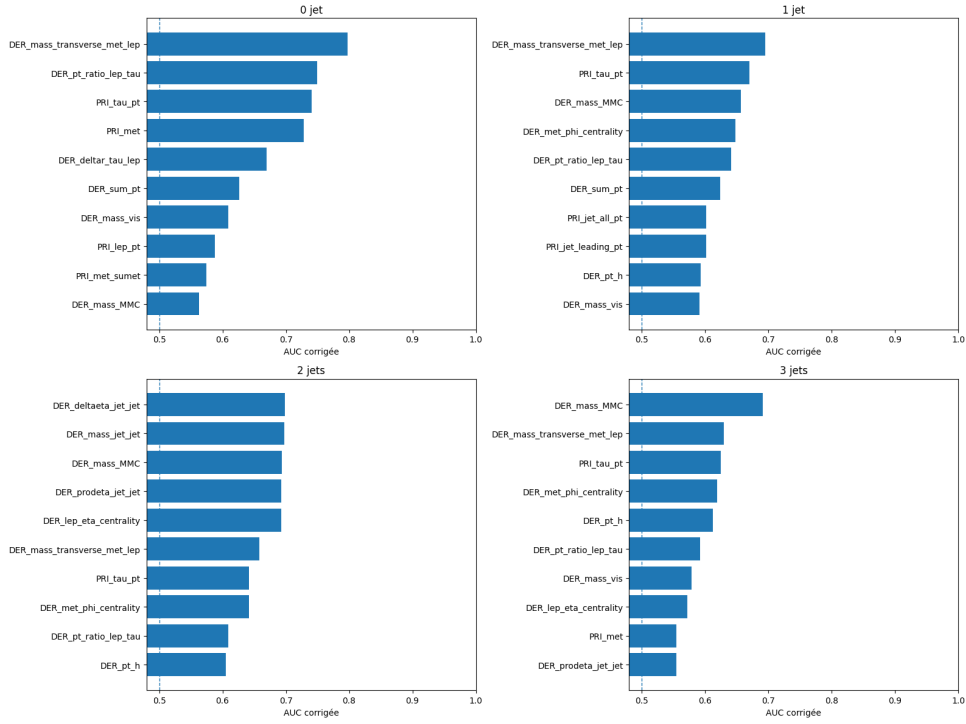


FIGURE 1 – Importance des colonnes suivant le nombre de jets

2.7 Analyse des corrélations

Après l'étude individuelle des variables, une analyse des dépendances entre variables a été réalisée. Pour cela, la corrélation de Spearman a été utilisée. Contrairement à la corrélation de Pearson, la corrélation de Spearman ne suppose pas de relation strictement linéaire entre variables. Ce choix est plus adapté dans le contexte des données physiques où les relations peuvent être non linéaires.

L'objectif de cette étape est d'identifier les variables redondantes, les groupes de variables portants des informations similaires, et les dépendances importantes entre grandeurs physiques.

Les matrices de corrélation obtenues montrent que certaines variables sont fortement liées entre elles, notamment les variables décrivant des quantités cinématiques proches. Cette analyse est importante pour l'interprétation future des modèles. En effet, deux variables très corrélées peuvent porter une information physique similaire. Dans un modèle de machine learning, leur importance peut donc être répartie entre elles.

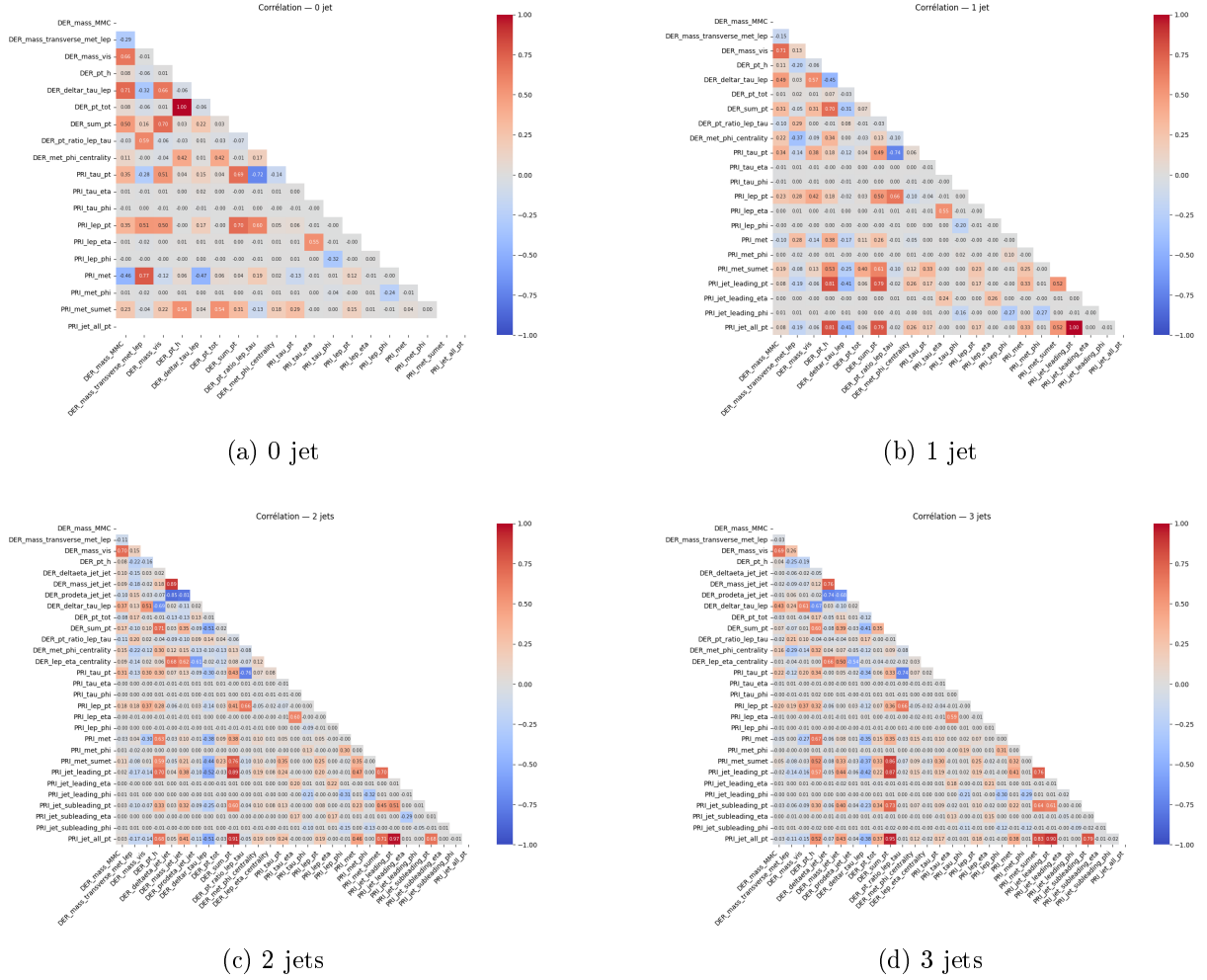


FIGURE 2 – Matrice de corrélation des variables selon le nombre de jets

2.8 Croisement entre corrélation et pouvoir discriminant

Afin d'aller plus loin dans l'analyse exploratoire, le pouvoir discriminant des variables a été comparé à leur niveau de corrélation. Cette approche permet de distinguer plusieurs situations :

- Une variable fortement discriminante et peu corrélée apporte une information particulièrement intéressant.
- Une variable fortement discriminante mais très corrélée peut être partiellement redondante avec d'autres.
- Une variable faiblement discriminante seule peut néanmoins devenir utile lorsqu'elle est combinée avec d'autres variables

Cette analyse montre que la séparation entre signal et bruit repose principalement sur une combinaison de plusieurs variables physiques plutôt que sur une seule grandeur isolée.

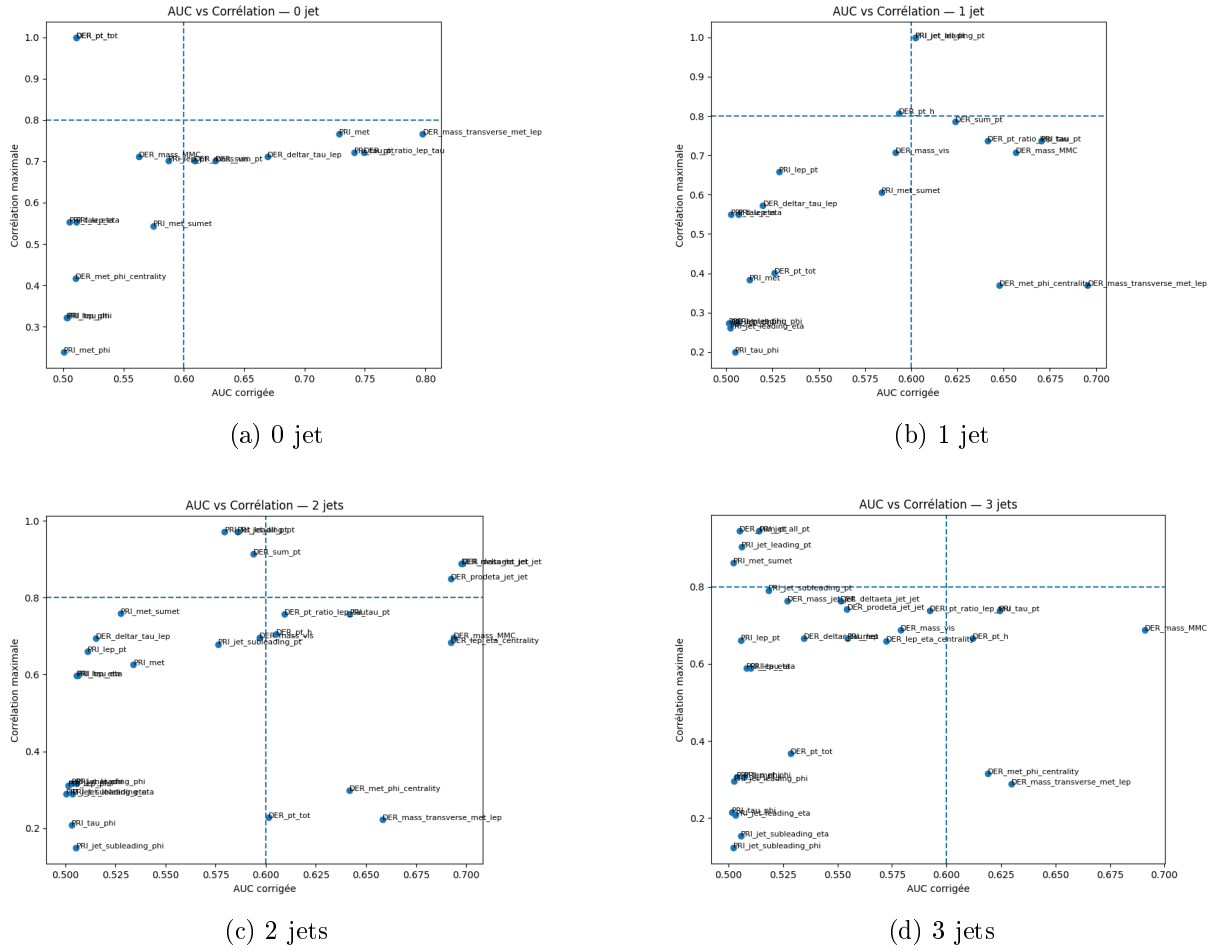


FIGURE 3 – Graphique AUC vs Corrélation suivant le nombre de jets

2.9 Analyse de la colonne *Weight* et rôle des poids statistiques

Contrairement à un problème classique de machine learning où toutes les observations possèdent la même importance, chaque événement du dataset Higgs est associé à un poids statistique (*Weight*).

Ces poids proviennent du fait que les données utilisées sont des données simulées par Monte-Carlo et ne représentent pas directement les proportions physiques réelles observées dans les collisions du LHC.

Dans la réalité expérimentale, les événements associés au boson de Higgs sont extrêmement rares par rapport au bruit de fond. Si les données étaient conservées dans leurs proportions physiques réelles, le signal deviendrait presque invisible pour les modèles d'apprentissage. Les organisateurs du challenge ont donc artificiellement augmenté la proportion d'événements signal afin de faciliter l'apprentissage du modèle. Les poids permettent ensuite de corriger cette surreprésentation artificielle afin de retrouver la contribution physique attendue de chaque événement.

Ainsi, les proportions brutes du datasets servent principalement à l'apprentissage, et les poids permettent de reconstruire les proportions physiques réelles.

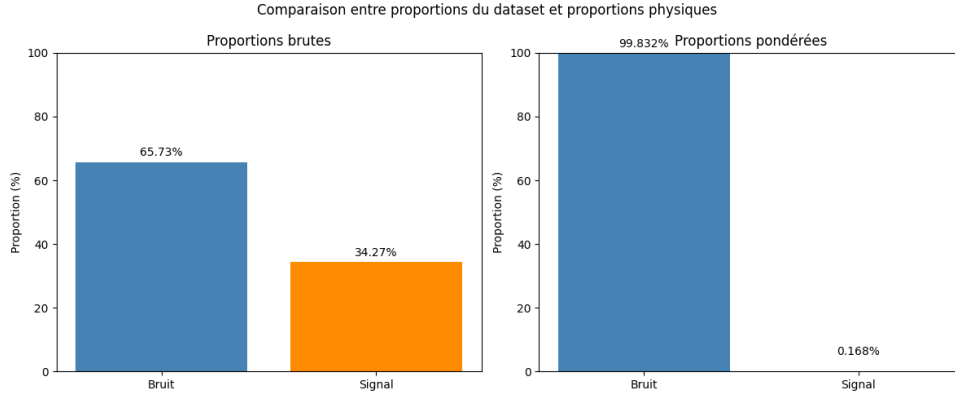


FIGURE 4 – Graphe de comparaison entre proportions du datasets et proportions physiques

Le graphique comparant proportions brutes et proportions pondérées met clairement en évidence cette différence. Dans le dataset d'apprentissage, le signal semble relativement fréquent. En revanche, après pondération, le signal devient extrêmement rare face au bruit de fond. Cette observation montre que le problème physique réel est beaucoup plus difficile que ne le suggèrent les simples effectifs du dataset.

2.10 Interprétation physique

Cette différence importante entre proportions brutes et proportions pondérées possède une interprétation physique directe. Le dataset fourni pour l'apprentissage a été volontairement rééquilibré afin que le modèle puisse apprendre des caractéristiques du signal. Sans ce rééquilibrage, le bruit de fond dominerait presque totalement les données. Cependant, dans les conditions physiques réelles, les événements de bruit sont extrêmement nombreux, et les événements associés au boson de Higgs sont très rares. Les poids permettent donc de relier le problème de classification à la réalité physique expérimentale.

Cette distinction entre proportions utilisées pour l'apprentissage et proportions physiques réelles constitue un aspect fondamental du challenge Higgs.

2.11 Impact sur l'évaluation des modèles

L'existence des poids statistiques implique qu'une erreur de classification n'a pas toujours le même impact physique. Par exemple, une erreur sur un événement faiblement pondéré possède un impact limité, tandis qu'une erreur sur un événement fortement pondéré peut grandement influencer l'évaluation physique du modèle.

Ainsi, une bonne accuracy classique ne garantit pas nécessairement une bonne capacité de détection du boson de Higgs. Cela justifie l'utilisation d'une métrique physique adapté : l'AMS (Approximate Median Significance)

2.12 Justification de l'AMS

Dans le contexte du Higgs, l'objectif n'est pas uniquement de bien classer les événements, mais surtout de détecter un excès statistiquement significatif de signal par rapport au bruit.

Cette métrique mesure la significativité statistique du signal extrait par le modèle. Ainsi, un modèle peut posséder une bonne accuracy mais une faible AMS et inversement. L'AMS est donc plus adaptée au contexte physique du problème que les métriques classiques de classification

2.13 Conclusion de l'exploration des données

L'exploration des données a permis de mettre en évidence plusieurs caractéristiques importantes du problème Higgs. L'analyse des valeurs manquantes a montré que celles-ci possèdent une signification physique et dépendent fortement du nombre de jets présents dans les événements. Cette observation a conduit à séparer les données selon `PRI_jet_num`, permettant de travailler sur des groupes physiquement plus homogènes.

L'étude des AUC univariées a permis d'identifier plusieurs variables fortement discriminantes, tandis que l'analyse des corrélations a mis en évidence des dépendances importantes entre certaines grandeurs physiques.

L'ensemble de ces résultats montre que la séparation entre signal et bruit repose principalement sur une combinaison complexe de plusieurs variables physiques. Ces observations suggèrent qu'une partie de l'information utile pourrait être mieux exploitée à travers la création de nouvelles variables combinant plusieurs grandeurs physiques. Une étape de feature engineering a donc été envisagée afin d'exploiter plus efficacement les interactions entre variables et d'améliorer la capacité des modèles à distinguer signal et bruit.

3 Feature Engineering

3.1 Importance du feature engineering

Le feature engineering est l'une des étapes les plus importantes d'un pipeline de machine learning en physique des particules. Les variables brutes du détecteur (pseudo-rapacité η et l'angle azimutal ϕ , impulsion transverse p_T) décrivent chaque particule individuellement, mais les modèles à arbres comme XGBoost travaillent mieux avec des variables qui capturent des relations entre particules (invariants de Lorentz, distances angulaires, masses invariantes).

Deux effets concrets justifient cet investissement. Premièrement, les modèles à arbres peu profonds ($\text{depth} = 6$) ne peuvent apprendre que des interactions de faible ordre : une variable comme `DER_tau_pt2_over_etot` rend la structure quadratique explicite sans nécessiter de profondeur supplémentaire. Deuxièmement, les features physiques réduisent le nombre d'arbres nécessaires, accélérant l'entraînement tout en améliorant la généralisation.

3.2 Explication des nouvelles colonnes

- **DER_énergie_totale** : Somme des impulsions transverses (p_T) du τ , du lepton et de l'énergie manquante. Elle quantifie l'énergie transverse total apparente, agissant comme un discriminateur clé entre le signal $H \rightarrow \tau\tau$ et les bruits de fond $Z/W + \text{jets}$.
- **DER_mass_transverse_lep_met** : Analogie de la masse transverse du boson W en physique hadronique. Dans les événements $W + \text{jets}$ (bruit de fond principal), cette masse forme un pic de Jacobi à environ 80 GeV. Dans le signal Higgs, la distribution est plus diffuse car l'énergie manquante provient de deux neutrinos, pas d'un seul.

- **DER_distance_tau_lepton** : Mesure la séparation géométrique $\Delta R = \sqrt{\Delta\eta^2 + \Delta\phi^2}$ entre le τ et le lepton dans l'espace (η, ϕ) . Dans la désintégration $H \rightarrow \tau\tau$, les produits tendent à être émis dos-à-dos ($\Delta R \sim \pi$) dans le référentiel du Higgs. Cette variable permet de caractériser la topologie de l'événement et de distinguer le signal des bruits de fond ayant des distributions de masse et d'ouverture angulaire différentes, comme le processus $Z \rightarrow \tau\tau$.

- **DER_pt_ratio_tau_lep** : Rapport d'impulsion transverse défini par p_T^τ/p_T^ℓ . Cette variable capture l'asymétrie énergétique entre les deux produits de désintégration. Dans le canal $H \rightarrow \tau_{had}\tau_{lep}$, ce ratio dépend des angles de désintégration et de la masse du Higgs. Elle est complétée par la feature A10 (asymétrie normalisée), qui s'avère plus stable numériquement pour le modèle.

- **DER_pt_times_met** : Interaction produit entre les deux variables les plus discriminantes. L'AMS du challenge Higgs est très sensible aux événements avec à la fois un tau énergétique et beaucoup d'énergie manquante, typique du signal où deux neutrinos emportent de l'énergie.

- **DER_abs_delta_eta** : Version absolue de la séparation en pseudo-rapidité, définie par la formule :

$$\text{DER_abs_delta_eta} = |\eta_\tau - \eta_\ell|$$

Cette variable est complémentaire au ΔR car elle isole la composante longitudinale de la séparation angulaire. Elle s'avère particulièrement sensible à la topologie VBF (*Vector Boson Fusion*) dans laquelle les jets forward induisent une asymétrie caractéristique en η .

- **DER_mass_inv_tau_lep** : Approximation collinéaire de la masse invariante du système tau-lepton (particules supposées de masse nulle), définie par la formule :

$$\text{DER_mass_inv_tau_lep} = \sqrt{2 \cdot p_T^\tau \cdot p_T^\ell \cdot (\cosh(\Delta\eta) - \cos(\Delta\phi))}$$

C'est l'une des variables les plus discriminantes : dans $H \rightarrow \tau\tau$, cette masse est directement corrélée à la masse du Higgs (~ 125 GeV), tandis que dans $Z \rightarrow \tau\tau$ elle est corrélée à ~ 91 GeV.

- **DER_pt_visible_higgs** : Somme vectorielle 2D des impulsions transverses du tau et du lepton, définie par la formule :

$$\text{DER_pt_visible_higgs} = \sqrt{(p_T^\tau \cdot \cos \phi_\tau + p_T^\ell \cdot \cos \phi_\ell)^2 + (p_T^\tau \cdot \sin \phi_\tau + p_T^\ell \cdot \sin \phi_\ell)^2}$$

Mesure le "boost" transverse du candidat Higgs visible. Un Higgs de haute masse produit via ggF (fusion de gluons) tend à avoir un plus grand p_T que les processus Z/W + jets de bruit de fond.

- **DER_pt_asymmetry** : Version normalisée du ratio A4, définie par la formule :

$$\text{DER_pt_asymmetry} = (p_T^\tau - p_T^\ell)/(p_T^\tau + p_T^\ell + \epsilon) \in [-1, +1]$$

Beaucoup plus stable numériquement (bornée dans $[-1, 1]$) et évite les divergences quand $p_T^\ell \rightarrow 0$. Capture la même asymétrie mais sous une forme que les arbres peu profonds peuvent apprendre plus efficacement avec des coupures symétriques.

- **DER_pt_imbalance** : Déséquilibre p_T vectoriel total, défini par la formule :

$$\text{DER_pt_imbalance} = |\vec{p}_T^\tau + \vec{p}_T^\ell + \vec{p}_T^{\text{MET}}|$$

La conservation de l'impulsion transverse impose que la somme vectorielle de toutes les particules visibles et invisibles soit nulle. Ce déséquilibre résiduel trahit des jets non mesurés, des ISR (*Initial State Radiation*) mal reconstruits, ou du bruit de fond QCD avec des mesures imparfaites.

- **DER_eta_centrality_sum** : Centralité totale en pseudo-rapidité, définie par la formule :

$$\text{DER_eta_centrality_sum} = |\eta_\tau| + |\eta_\ell|$$

Les événements Higgs produits via fusion de gluons (ggH) tendent à avoir des leptons centraux ($|\eta|$ petit) car les gluons interagissent dans la région centrale du détecteur. Le bruit de fond QCD pousse les leptons vers les régions *forward* (grand $|\eta|$). Variable simple mais efficace.

- **DER_met_over_mass_vis** : Énergie manquante relative à la masse visible, définie par la formule :

$$\text{DER_met_over_mass_vis} = \text{PRI_met} / (\text{DER_mass_vis} + \epsilon)$$

Dans $H \rightarrow \tau\tau$, les deux taus se désintègrent chacun en émettant un neutrino. Le rapport énergie manquante / masse visible mesure la fraction d'énergie "cachée". Un ratio élevé indique que les neutrinos emportent une grande fraction de l'énergie totale, très typique du signal.

- **DER_dphi_tau_met** : Angle azimutal $\Delta\phi$ entre le τ et la MET, défini par la formule :

$$\text{DER_dphi_tau_met} = |\phi_\tau - \phi_{\text{MET}}|$$

(corrigé pour la circularité : valeur $\in [0, \pi]$). Dans $H \rightarrow \tau_{had}\tau_{lep}$, le neutrino issu du tau hadronique est colinéaire à celui-ci (approximation colinéaire). L'angle $\Delta\phi(\tau, \text{MET})$ est donc petit pour le signal. Pour le bruit W + jets, la corrélation entre le lepton et le MET est différente, offrant une discrimination supplémentaire.

- **DER_jet_pt_fraction** : Fraction p_T du jet *leading*, définie par la formule :

$$\text{DER_jet_pt_fraction} = p_T^{jet1} / (p_T^\tau + p_T^\ell + p_T^{jet1} + \epsilon)$$

Mesure la fraction de l'énergie transverse totale portée par le jet le plus énergétique. Cette valeur est élevée dans les événements W + jets/ $t\bar{t}$ où le jet domine, et faible dans le signal $H \rightarrow \tau\tau$ sans jet (régime ggH). Complémentaire à **PRI_jet_num**, elle apporte une nuance énergétique que la variable discrète ne capture pas.

3.3 Data Preprocessing : nettoyage et séparation des données

Dans cette partie, nous avons uniquement transformé les étiquettes s et b en valeurs numériques 1 et 0 (*label encoding*) afin que le modèle puisse les traiter.

Par ailleurs, les valeurs manquantes (identifiées par la valeur -999.0) ont été laissées telles quelles dans le jeu de données, car l'algorithme XGBoost que nous allons utiliser possède une capacité native à gérer ces entrées sans nécessiter de transformation. Dans l'éventualité de l'utilisation d'autres modèles, il conviendrait d'analyser l'impact du traitement de ces valeurs manquantes, car leur présence peut apporter une information physique discriminante.

De plus, nous n'avons pas besoin de normaliser nos valeurs car XGBoost s'appuie sur des arbres de décision. Ces modèles sont invariants aux transformations monotones des variables d'entrée, ce qui signifie qu'ils traitent les données en fonction de leur ordre relatif plutôt que de leur échelle absolue.

Afin de faciliter l'exploration dans différents notebooks, nous avons modularisé le prétraitement dans un script Python externe. Nous avons ainsi développé une fonction `load_and_process_data()`, qui renvoie directement les dataframes dont nous aurons besoin :

```
X, y, w, X_test, id_test = load_and_process_data()
```

Cette fonction centralise l'ensemble des étapes (chargement, nettoyage, encodage et feature engineering), ce qui permet d'obtenir immédiatement des données prêtes pour l'entraînement dès l'ouverture d'un nouveau notebook, sans avoir à réécrire ou réexécuter manuellement chaque étape de préparation.

4 Modèle, Seuil et Optimisation

4.1 Explication des modèles

4.1.1 Choix du type de modèle

Pour traiter les données de ce challenge, nous avons choisi d'utiliser des algorithmes de Gradient Boosting sur arbres de décision. Ce choix repose sur plusieurs arguments liés à la nature de notre jeu de données :

D'abord, ces modèles sont particulièrement performants sur les données tabulaires. Comme nous travaillons sur des mesures physiques structurées, le Boosting d'arbres permet de capturer des relations complexes et non-linéaires entre les variables sans nécessiter un prétraitement lourd des données.

Le principe du Gradient Boosting est une méthode d'apprentissage dit « d'ensemble ». L'idée est de combiner plusieurs modèles simples (des arbres de décision peu profonds) de manière itérative. Au lieu de construire un seul modèle parfait, on ajoute des arbres les uns après les autres : chaque nouvel arbre est entraîné pour corriger les erreurs de prédiction laissées par les précédents.

4.1.2 XGBoost

XGBoost (eXtreme Gradient Boosting) est sans doute le modèle le favori pour ce projet, car il a été initialement développé et popularisé lors de ce challenge sur Kaggle. Ce qui fait sa force par rapport aux autres algorithmes, c'est sa gestion très fine de l'overfitting. Il intègre des fonctions de régularisation mathématique qui obligent les arbres à rester simples, ce qui lui permet de mieux prédire des données qu'il n'a jamais vues. De plus, sa capacité à traiter les valeurs manquantes sans aucune préparation préalable en fait un outil extrêmement robuste et fiable pour les physiciens.

Voici une explication des principaux hyperparamètres, dont certains seront optimisés par la suite :

- **n_estimators** : C'est le nombre total d'arbres que l'on va construire. Plus il y en a, plus le modèle est complexe. S'il est trop élevé, on risque l'overfitting. Nous le fixerons à 5000 en utilisant un early stopping à 100.
- **max_depth** : Il définit la profondeur maximale de chaque arbre. Un arbre trop profond mémorisera trop les détails du bruit de fond, alors qu'un arbre court sera plus généraliste. Nous essaierons des combinaisons avec une profondeur de 5 ou 6.
- **learning_rate** : C'est le pas d'apprentissage. Il réduit l'importance de chaque nouvel arbre pour avancer plus prudemment vers la solution optimale et améliorer la stabilité du modèle. Un learning rate trop faible risque de ralentir excessivement la convergence de l'algorithme, tandis que s'il est trop élevé, nous passerons sûrement à côté du minimum global.
- **min_child_weight** : Il définit la quantité minimale de données requise pour créer un nouveau nœud dans l'arbre. Plus il est élevé, plus l'algorithme est conservateur face au bruit.
- **subsample** : C'est le pourcentage de données utilisées pour entraîner chaque arbre. En prenant une partie des données au hasard, on rend le modèle plus robuste.
- **colsample_bytree** : Similaire au *subsample*, mais pour les variables physiques. On ne donne qu'une partie des colonnes à chaque arbre pour éviter qu'il ne dépende trop d'une seule variable.
- **reg_alpha** (L1) et **reg_lambda** (L2) : Ce sont les paramètres de régularisation. Ils ajoutent une pénalité mathématique si les arbres deviennent trop complexes, forçant le modèle à rester simple afin d'éviter l'overfitting.
- **tree_method** : C'est la méthode de construction des arbres. L'utilisation de la méthode *hist* permet d'accélérer considérablement les calculs sur de gros jeux de données comme le notre.

4.1.3 CatBoost

CatBoost (pour *Categorical Boosting*) est un algorithme de *Gradient Boosting*. Son principal avantage est qu'il donne souvent de très bons résultats sans même avoir besoin de modifier ses réglages de base. Sa particularité est d'utiliser des « arbres symétriques » : à chaque étage de l'arbre, la question posée est la même pour tous les nœuds. Cette structure est assez rigide, ce qui empêche naturellement le modèle d'apprendre par cœur les données. Même si nous avons remarqué qu'il est plus lent à s'entraîner que les autres, il reste très stable.

Voici une explication des principaux hyperparamètres, dont certains seront optimisés par la suite :

- **iterations** : C'est le nombre maximal d'arbres à construire. Nous le fixons à 5000 en utilisant un early stopping à 100.
- **depth** : La profondeur des arbres. Comme les arbres sont symétriques, une profondeur de 3 à 7 est généralement suffisante pour capturer les interactions complexes sans exploser le temps de calcul.
- **learning_rate** : Le pas d'apprentissage, comme précédemment.
- **l2_leaf_reg** : Ce paramètre contrôle la régularisation L2 sur les feuilles de l'arbre. Il est essentiel pour éviter que le modèle ne colle trop précisément aux poids du signal dans le jeu d'entraînement.
- **bootstrap_type** et **subsample** : Avec le mode *Bernoulli*, nous sélectionnons une fraction aléatoire des données pour chaque étape de boosting, ce qui diversifie l'apprentissage et renforce la robustesse.
- **random_strength** : Ce paramètre propre à CatBoost ajoute du caractère aléatoire lors de

- l'évaluation des séparations des arbres, aidant ainsi à lutter contre le surapprentissage.
- `eval_metric` : Nous suivons l'*AUC* (Aire sous la courbe ROC) pendant l'entraînement, car cette métrique est plus stable que l'AMS pour guider l'early stopping.

4.1.4 LightGBM

LightGBM (*Light Gradient Boosting Machine*) a été conçu pour être extrêmement rapide et efficace en mémoire, ce qui est un avantage majeur vu la taille de notre jeu de données. Sa grande particularité réside dans sa façon de construire les arbres : au lieu de faire grandir l'arbre étage par étage (*level-wise*), il le fait grandir par feuilles (*leaf-wise*). Il choisit de diviser la feuille qui réduit le plus l'erreur, peu importe si l'arbre devient déséquilibré. Cela lui permet d'être souvent plus précis et beaucoup plus rapide que les autres algorithmes de boosting.

Voici une explication des principaux hyperparamètres, dont certains seront optimisés par la suite :

- `objective` : Définit la tâche du modèle. Nous utilisons *binary* car nous cherchons à classer les données en deux catégories (Signal ou Bruit).
- `metric` : La mesure utilisée pour guider l'apprentissage. L'*AUC* est choisie ici car elle évalue la qualité du classement global des probabilités.
- `boosting_type` : Le type d'algorithme. *gbdt* est la méthode classique qui construit les arbres de manière séquentielle.
- `n_estimators` : Le nombre maximum d'arbres que le modèle peut construire (fixé ici à 2500).
- `learning_rate` : Le pas d'apprentissage, comme précédemment.
- `num_leaves` : Le nombre maximal de feuilles par arbre. C'est le paramètre principal de LightGBM pour contrôler la complexité du modèle.
- `max_depth` : Limite de la profondeur des arbres
- `min_child_samples` : Nombre minimum d'échantillons requis dans une feuille. Cela sert de sécurité pour ne pas créer de règles basées sur trop peu de données.
- `subsample` et `subsample_freq` : Définissent la fraction de données utilisée pour chaque arbre et la fréquence de renouvellement de cet échantillon. Cela permet de varier l'apprentissage.
- `colsample_bytree` : Pourcentage de variables physiques sélectionnées au hasard pour chaque arbre, ce qui empêche le modèle de trop se reposer sur une seule variable dominante.
- `reg_alpha` et `reg_lambda` : Paramètres de régularisation L1 et L2 qui pénalisent les modèles trop complexes pour favoriser la généralisation.

4.2 Optimisation du seuil

4.2.1 Explication du seuil

Une fois que notre modèle a fini d'apprendre, il ne nous renvoie pas une réponse "signal" ou "bruit", mais des probabilités d'appartenance à la classe signal. Par défaut, beaucoup d'algorithmes coupent à 0.5, mais pour ce challenge, ce seuil n'est pas idéal. Nous avons donc choisi d'utiliser un seuil basé sur un percentile. C'est-à-dire que nous allons classer toutes nos prédictions de la plus probable à la moins probable, et ne sélectionner que les $X\%$ des événements ayant obtenu les scores les plus élevés pour les déclarer comme "signal".

Cette approche est beaucoup plus robuste qu'un seuil fixe à 0.5 pour deux raisons principales :

- **Déséquilibre des classes** : Dans ce challenge, le signal est beaucoup plus rare que le bruit de fond. En fixant un percentile (par exemple le Top 15 %), on s'assure de soumettre un volume de signal cohérent avec la réalité physique de l'expérience, peu importe si les probabilités brutes du modèle sont centrées sur 0.2 ou 0.8.
- **Stabilité de l'AMS** : Comme le montre le graphique d'optimisation ci dessous, le score AMS dépend énormément de la quantité d'événements sélectionnés. En utilisant un percentile, on stabilise le modèle : même si on change d'algorithme, on garde la même proportion d'événements classés en signal, ce qui permet de comparer les performances des modèles sur une base équitable.

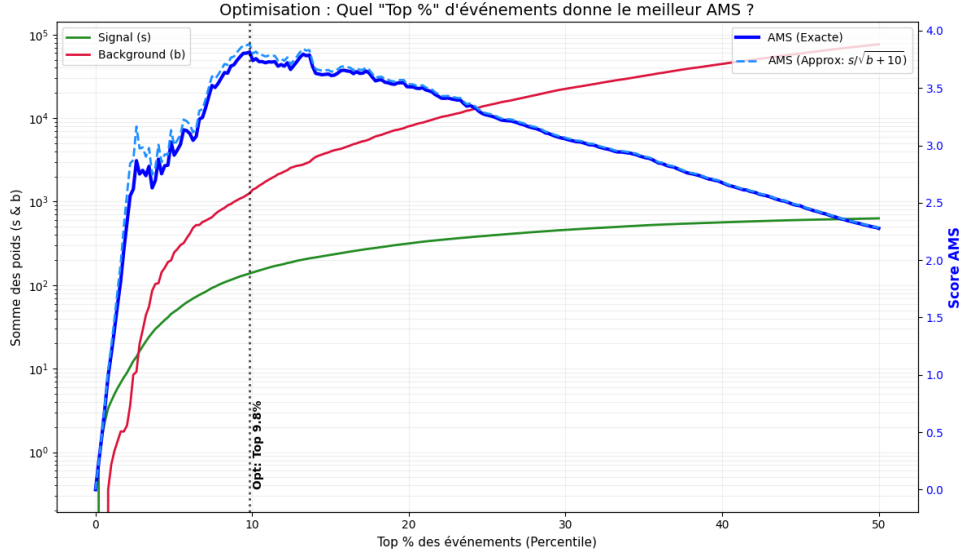


FIGURE 5 – Optimisation du seuil pour maximiser l'AMS

On voit sur ce graphique que l'AMS est un score instable. Il est à noter également que l'échelle de la somme des poids est logarithmique.

L'analyse de la courbe de distribution des poids permet de mieux comprendre la structure de la formule de l'AMS. On remarque que la courbe du signal démarre de façon très abrupte, tandis que celle du bruit de fond ne commence à croître significativement que plus tard. C'est précisément ce décalage qui justifie le terme de régularisation (+10) dans le dénominateur de la formule : sans cette constante, le score AMS serait artificiellement "boosté" au tout début du graphique, là où le bruit est quasi nul. La constante force ainsi le modèle à sélectionner un volume de données suffisant pour être statistiquement significatif, avant que l'accumulation massive du bruit ne finisse par faire chuter le score.

Par la suite, nous allons lancer de nombreuses simulations via Optuna en maximisant systématiquement l'AMS. L'objectif est de vérifier s'il existe un percentile qui se dégage empiriquement comme étant le plus performant. Cette approche nous permettra de fixer un seuil robuste, capable de bien généraliser sur de nouvelles données. Nous effectuerons 4 à 5 folds suivant les modèles pour limiter la variance.

4.2.2 XGBoost

Nous avons effectué 5 folds et 200 trials afin d'afficher une tendance en limitant la variance, en sachant qu'XGBoost est le modèle le plus prometteur. Ce graphique révèle que les essais d'optimisation du seuil n'ont pas une distribution aléatoire, mais forme une courbe de tendance en forme de cloche. Cette structure confirme l'existence d'un optimum global pour le score AMS aux

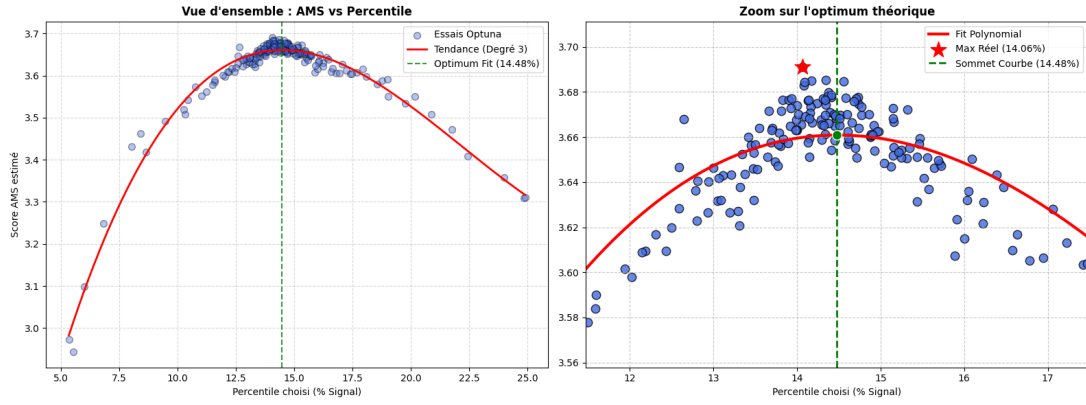


FIGURE 6 – Optimisation Seuil XGBoost

alentours d'un percentile de 14.5%. On observe toutefois que la zone située au sommet de la courbe est caractérisée par une forte volatilité : la dispersion des points s'accroît nettement au-delà de 3.60. Cette instabilité illustre la sensibilité de la métrique AMS aux fluctuations statistiques dans les hautes performances, où de légères variations dans la classification des événements peuvent induire des écarts de scores significatifs. En prenant le sommet du fit polynomial de degré 3, nous garantissons une sélection de seuil plus robuste face à ce bruit statistique.

4.2.3 CatBoost

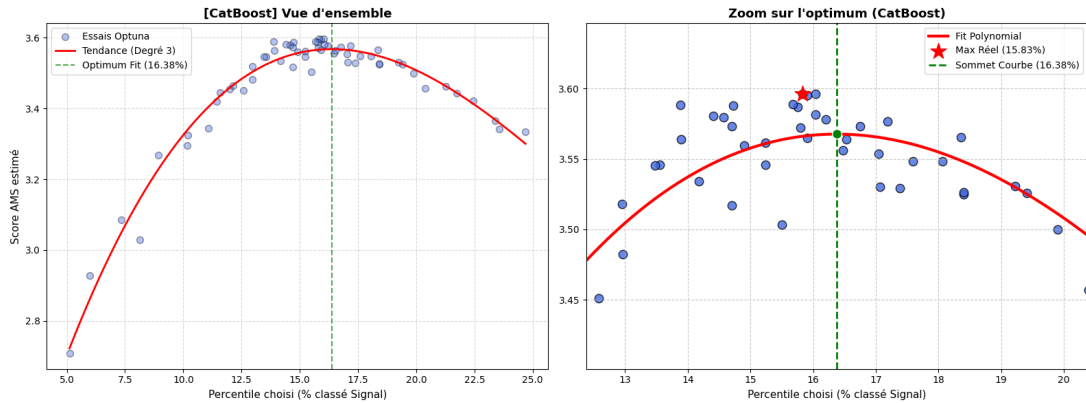


FIGURE 7 – Optimisation Seuil CatBoost

Nous avons effectué 4 folds et 60 trials pour CatBoost, car chaque trial était très long. Dans le cas du CatBoost, nous observons cette même courbe de tendance en forme de cloche. À la suite du fit polynomial de degré 3, nous voyons que le sommet se situe à un percentile de 16,4%. Nous avons néanmoins pris un percentile plus faible, au vu de la concentration de score plus élevé sur la gauche du sommet, en prenant 16%.

4.2.4 LightGBM

LightGBM étant plus rapide que CatBoost, nous avons effectué 4 folds et 100 trials. Pour LightGBM, nous remarquons une plus grande variance au niveau des points, mais une tendance en forme de cloche se forme néanmoins. Nous pouvons donc opter pour un percentile à 15,2%.

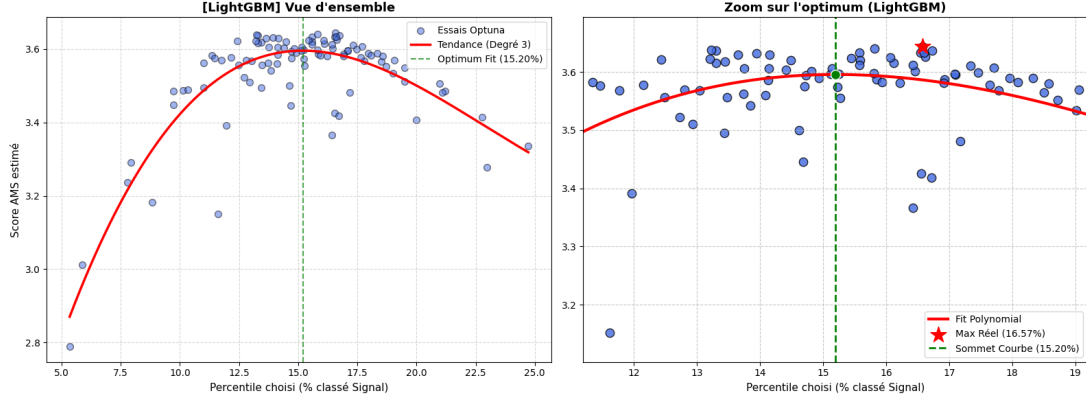


FIGURE 8 – Optimisation Seuil LightGBM

4.3 Optimisation des hyperparamètres des modèles

Afin d’affiner les performances de nos modèles, nous allons utiliser Optuna, qui est une bibliothèque d’optimisation automatique particulièrement performante. Contrairement à une recherche exhaustive de type Grid Search, Optuna utilise l’optimisation bayésienne.

Cette approche est plus efficace car elle ne teste pas les combinaisons de paramètres au hasard : elle apprend des essais précédents pour orienter les recherches futures vers les zones de l’espace des paramètres les plus prometteuses.

Afin de mieux comprendre la structure de l’espace de recherche exploré par Optuna, nous avons appliqué l’algorithme *UMAP* sur l’ensemble des essais effectués. Cette technique de réduction de dimension non-linéaire nous a permis de projeter les combinaisons d’hyperparamètres sur un plan en 2D, tout en préservant au maximum les relations de proximité topologique. L’objectif de cette visualisation est d’identifier visuellement des « régions de haute performance » : en colorant les points selon leur score AMS, nous pouvons vérifier si les meilleurs modèles sont regroupés dans des zones spécifiques ou s’ils sont isolés. Cette approche permet de confirmer la convergence de l’optimisation bayésienne et d’assurer que le modèle ne survit pas sur un optimum local instable, mais s’inscrit dans une zone de paramètres robuste et cohérente.

4.3.1 XGBoost

Pour XGBoost, nous utiliserons 5 folds, 200 trials et un percentile à 14.5%. Nous obtenons les hyperparamètres suivant :

Hyperparamètre	Valeur	Hyperparamètre	Valeur
max_depth	5	reg_alpha	1.78×10^{-7}
learning_rate	0.00681	reg_lambda	4.07×10^{-5}
min_child_weight	8	subsample	0.87872
colsample_bytree	0.54181	n_estimators	2664.0
Score AMS (OOF) : 3.7076			

TABLE 1 – Hyperparamètres optimaux - Trial 108

Nous remarquons que certains hyperparamètres sont proche de leur borne basse ou haute. Afin d’améliorer encore notre score, nous aurions pu ajuster les bornes à chaque fin d’étude. Nous avons décidé de ne pas le faire car l’entraînement prenait plusieurs heures, et que les améliorations sur le score étaient très faible.

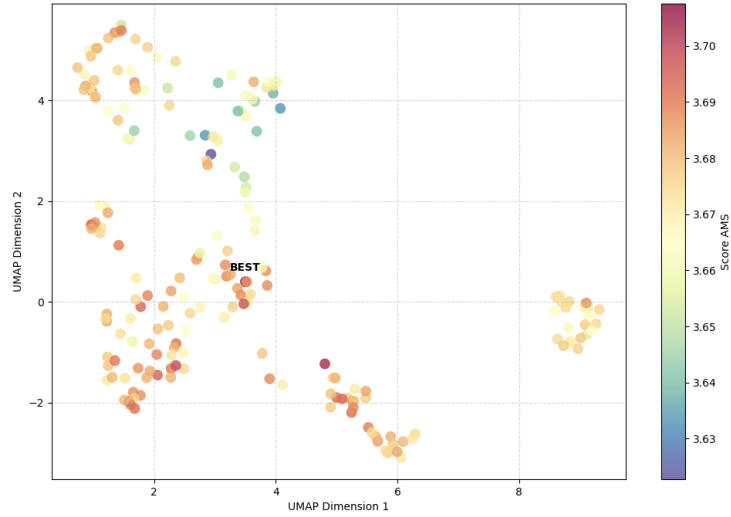


FIGURE 9 – UMAP XGBoost

Nous voyons avec le graphique que notre meilleur score se situe dans un cluster de score élevé, montrant que notre solution optimale ne constitue pas un point isolé ou le fruit du hasard, mais s'inscrit au cœur d'un cluster de haute performance.

4.3.2 CatBoost

Pour CatBoost, nous utiliserons 4 folds, 50 trials et un percentile à 16%, car l'algorithme est très lent. Nous obtenons les hyperparamètres suivant :

Hyperparamètre	Valeur	Hyperparamètre	Valeur
depth	6	l2_leaf_reg	7.50005
learning_rate	0.05350	random_strength	1.25665
subsample	0.62402	n_estimators	4787.0
percentile	16%		
Score AMS (OOF) : 3.6218			

TABLE 2 – Hyperparamètres optimaux - Trial 41

Comme pour XGBoost, nous remarquons que certains hyperparamètres sont proches des bornes définies, notamment le nombre d'arbres. Au vu du temps d'entraînement et du score, nous avons décidé de garder notre modèle ainsi.

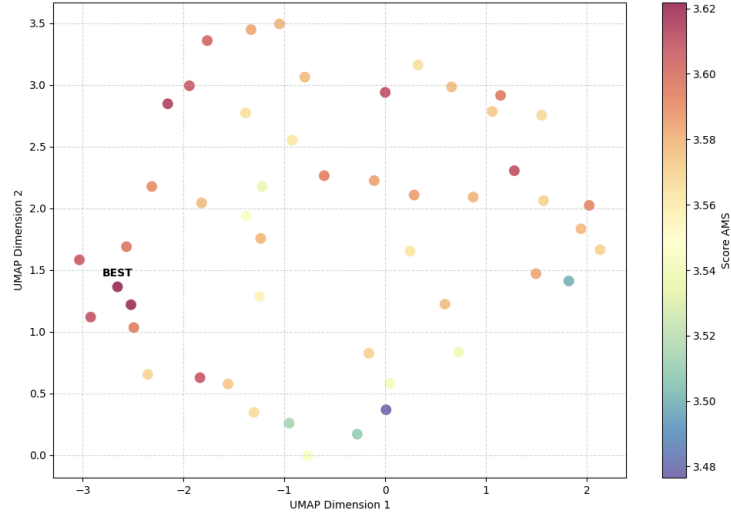


FIGURE 10 – UMAP CatBoost

Nous remarquons qu'il n'y a pas de cluster sur notre graphique, montrant donc que montrant ainsi que l'espace des hyperparamètres pour le modèle CatBoost est nettement moins structuré que celui de XGBoost.

4.3.3 LightGBM

Pour LoghtGBM, nous utiliserons 4 folds, 100 trials et un percentile à 15.2%. Nous obtenons les hyperparamètres suivant :

Hyperparamètre	Valeur	Hyperparamètre	Valeur
max_depth	4	reg_alpha	2.46822
learning_rate	0.02358	reg_lambda	0.06158
num_leaves	90	subsample	0.92944
min_child_samples	98	colsample_bytree	0.51575
n_estimators	2492.5	percentile	15.2%
Score AMS (OOF) : 3.6821			

TABLE 3 – Hyperparamètres optimaux - Trial 72 (LightGBM)

Nous remarquons pour LightGBM que le nombre d'arbres touche la limite supérieur. Nous avons également décidé de laisser le modèle ainsi au vu du temps d'entraînement et du score.

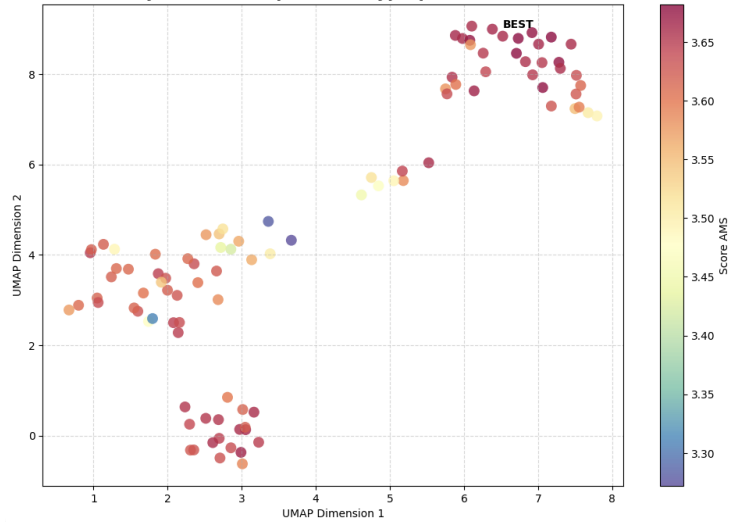


FIGURE 11 – UMAP LightGBM

La projection Umap pour LightGBM nous montre des résultats très concluant. Elle révèle une structure de l'espace des hyperparamètres extrêmement bien définie, avec plusieurs clusters, dont un de haute performance ou se situe notre meilleur score,confirmant la convergence de l'optimisation et garantit que nos paramètres sont situés dans une zone de résultats stables et non isolés.

4.3.4 Conclusion sur l'optimisation des hyperparamètres des modèles

Nous obtenons des scores corrects, malgré que certains de nos hyperparamètres sont très proches des bornes. Pour cela, nous aurions pu optimiser nos bornes, mais nous ne l'avons pas fait car l'entraînement des modèles était déjà très long.

4.4 Scores

4.4.1 Explication du score Kaggle

Le classement de la compétition s'appuie sur une partition des données de test en deux sous-ensembles distincts : le *Public Leaderboard* et le *Private Leaderboard*. Durant la phase d'entraînement, seul le score **Public**, calculé sur 18% des données de test, est visible par les participants pour orienter le réglage des modèles. Cependant, le classement final et définitif est établi à partir du score **Private**, calculé sur les 82% restants. Cette répartition asymétrique est conçue pour pénaliser les modèles qui s'ajusteraient trop précisément au hasard statistique de l'échantillon public, garantissant ainsi que la performance reflète une réelle capacité de généralisation sur les événements de collision produits par le Boson de Higgs. Le score **Private** aura donc plus d'importance.

4.4.2 XGBoost

Lors de la soumission finale, en prenant les meilleurs hyperparamètres ainsi qu'un percentile à 14.5, nous obtenons un score de 3.695 sur le **Private** et 3.614 sur le **Public**. C'est un score

légèrement en dessous de celui de l'entraînement, mais qui montre que le modèle a bien généralisé, car nous n'avons pas d'effondrement sur le **Private**.

4.4.3 CatBoost

Pour CatBoost, nous obtenons un score de 3.667 sur le **Private** et 3.602 sur le **Public** avec un percentile à 16. Ce score est plus élevé que celui obtenu pendant l'entraînement, et montre donc que le modèle a bien généralisé, sans pour autant dépassé le score du XGBoost.

4.4.4 LightGBM

Pour LightGBM, nous obtenons un score de 3.662 sur le **Private** et 3.673 sur le **Public** avec un percentile à 15.2. Ce score est moins élevé que celui durant l'entraînement, mais reste tout à fait cohérent.

4.4.5 Importances des colonnes pour le XGBoost

Afin de regarder les colonnes qui servent le mieux à séparer le signal du bruit, nous pouvons analyser l'importance des variables par le calcul du Gain, qui quantifie la contribution réelle de chaque caractéristique à la réduction de l'impureté au sein des arbres de décision. L'importance des variables est calculée ici via le critère du Gain. Pour chaque arbre du modèle, cet indicateur mesure l'amélioration de la précision (la réduction de l'erreur) apportée par chaque division (split) effectuée sur une variable donnée. Plus une variable permet de séparer nettement le signal du bruit de fond, plus son gain augmente.

Le score final représente la somme normalisée de ces gains sur l'ensemble des 2664 arbres du Trial 108 pour XGBoost. Contrairement à la simple fréquence d'utilisation (Weight), le gain privilégie les variables qui prennent les décisions les plus déterminantes. Cette approche permet d'identifier les variables ayant le plus fort impact sur la maximisation de l'AMS et de confirmer scientifiquement que le modèle s'appuie sur des caractéristiques physiques clés du Boson de Higgs plutôt que sur du bruit statistique.

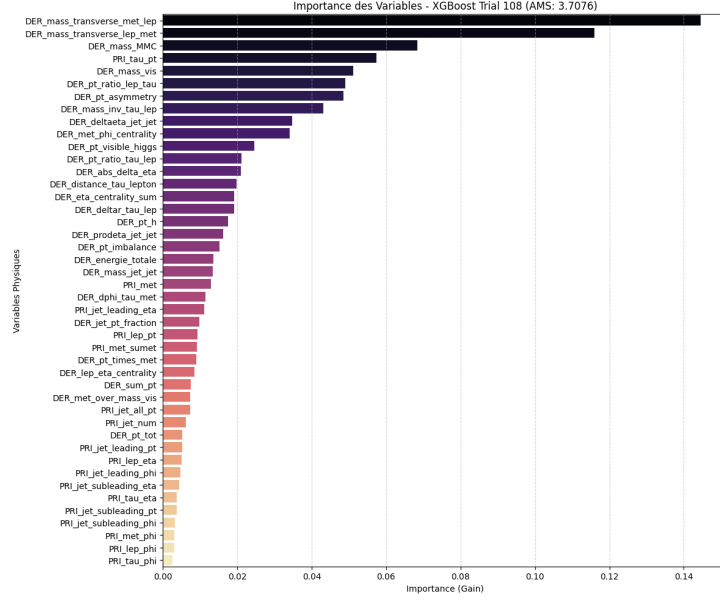


FIGURE 12 – Importance des variables pour XGBoost

A la lecture du graphique, on observe une redondance au sommet du classement : les deux variables dominantes sont quasi identiques. Ce doublon involontaire a finalement servi le modèle en lui offrant un signal plus direct pour maximiser son score. Ce résultat confirme la cohérence physique de l'apprentissage, puisque la masse s'impose comme le discriminant majeur pour identifier le boson de Higgs.

Nous remarquons que ces colonnes sont cohérentes avec celles trouvées précédemment lors de l'analyse du pouvoir discriminant des variables.

4.4.6 Conclusion sur les scores

Nous remarquons globalement que les scores sont cohérents avec ceux de l'entraînement, mais qu'ils sont assez volatils entre 3.65 et 3.70 car l'AMS est très sensible aux fluctuations statistiques. Nous avons privilégié une approche rigoureuse en optimisant systématiquement les hyperparamètres et le seuil de classification. Cette démarche empirique nous a permis d'éviter une recherche par tâtonnement.

4.5 Autres approches

Il existe d'autres approches que nous n'avons pas traité pendant ce projet qui nous aurait sûrement permis de gagner quelques centièmes sur le score final si elles étaient optimisées au maximum. Voici quelques une de ces approches.

4.5.1 Réseau de neurones

Nous avons également testé une approche axée sur le deep learning : un Perceptron Multi-Couches (MLP — Multilayer Perceptron). Son architecture est la suivante :

$\text{Input}() \rightarrow [\text{Linear} \rightarrow \text{BatchNorm} \rightarrow \text{SiLu} \rightarrow \text{Dropout}] \rightarrow N_{\text{couches}} \rightarrow \text{Sigmoid}$

où chaque bloc applique successivement une transformation linéaire, une normalisation par batch, une activation SiLU et un dropout régularisateur. Les hyperparamètres ont été optimisés via Optuna sur 8 trials. L'entraînement utilise une Binary Cross-Entropy pondérée par les poids événementiels Kaggle, normalisés dans chaque batch pour éviter que les événements à fort poids ne monopolisent le gradient. Nous avons ainsi obtenu un score de 3.66 en private score et 3.65 en public score sur Kaggle.

Troisièmement, contrairement à XGBoost qui gère nativement les structures tabulaires avec des coupures binaires adaptées, le MLP requiert un volume de données et un temps d'optimisation bien plus importants pour atteindre son plein potentiel sur ce type de données. Enfin, l'early stopping basé sur l'AMS de validation introduit une instabilité : l'AMS est une métrique bruitée sur un sous-ensemble de 20Au vu de ces résultats, et afin de concentrer nos efforts sur les approches les plus performantes, le MLP ne sera pas retenu dans la suite du projet.

4.5.2 Stacking de modèles

Bien que nos modèles individuels, notamment XGBoost et LightGBM, affichent des performances remarquables, une suite logique à ce travail aurait été la mise en œuvre d'une architecture en *stacking*. Cette méthode consiste à entraîner un méta-modèle pour combiner les prédictions de nos différents modèles de base. L'intérêt majeur de cette approche est de combler les faiblesses spécifiques de chaque algorithme : là où un modèle pourrait échouer à capturer certaines subtilités du signal, un autre pourrait s'avérer plus performant.

4.5.3 Décomposition suivant PRI_jet_num

Une autre piste d'amélioration a été d'effectuer une décomposition par nombre de jets (`PRI_jet_num` = 0, 1, ≥ 2). Cette approche présentait une motivation physique solide, les événements se distinguant effectivement par leur topologie selon le nombre de jets produits. Cependant, les résultats obtenus n'ont pas été concluants, les performances restant en deçà des attentes malgré la pertinence théorique de la démarche.

5 Autoencodeur

5.1 Ajout de colonnes supplémentaire

Pour l'Autoencodeur, nous allons rajouter des colonnes supplémentaires orientées jets et tau_pt.

- **DER_tau_pt_over_jet_sum** : Cette variable mesure la dominance énergétique du tau par rapport aux jets via le calcul $\frac{pT_{\text{tau}}}{pT_{\text{jet1}} + pT_{\text{jet2}} + pT_{\text{tau}} + \epsilon}$. Elle permet de distinguer le signal $H \rightarrow \tau\tau$, où le tau est très énergétique, des bruits de fond comme le $t\bar{t}$ où les jets dominent.
- **DER_tau_pt_times_jet_num** : Défini par le produit $pT_{\text{tau}} \times \text{PRI_jet_num}$, cette variable capture l'interaction cinématique entre l'énergie du tau et le régime de production

(VBF vs ggH). Elle rend directement accessible aux arbres de décision une corrélation qui nécessiterait autrement une plus grande profondeur de calcul.

- **DER_tau_over_jet_leading** : Rapport défini par $pT_{\tau}/(pT_{jet1} + \epsilon)$. Si le jet *leading* est plus énergétique que le tau, il s'agit souvent de bruit QCD. Dans le signal Higgs, le tau a tendance à avoir un pT comparable ou supérieur au jet d'hadronisation. Les cas sans jet produisent une valeur très élevée, agissant ainsi comme un indicateur cohérent de l'absence de jet (*missing*).
- **DER_dr_tau_jet1** : Distance géométrique entre le tau et le jet le plus énergétique, définie par $\sqrt{(\eta_{\tau} - \eta_{jet1})^2 + (\Delta\phi(\tau, jet1))^2}$. Dans la topologie VBF, les jets *forward* ($|\eta| > 2$) sont loin du tau central (ΔR grand). Dans les événements QCD avec un "faux-tau" (jet mal identifié comme tau), le jet et le tau candidat sont souvent proches (ΔR petit).
- **DER_tau_eta_centrality** : Défini par la relation $(\eta_{\tau} - \eta_{jet1})/(\eta_{jet2} - \eta_{jet1} + \epsilon) \in [0, 1]$ en topologie VBF. Dans la production VBF (*Vector Boson Fusion*), les deux jets *forward* "encadrent" le Higgs central. Si $\xi \in [0, 1]$, le tau est entre les jets en pseudo-rapacité — signature caractéristique du régime VBF. Cette variable est utilisée en physique expérimentale pour séparer les modes de production ggH et VBF.
- **DER_tau_pt2_over_etot** : Feature quadratique explicite définie par $pT_{\tau}^2/(pT_{\tau} + pT_{lep} + MET + pT_{jets} + \epsilon)$. Les modèles à arbres peu profonds ($depth = 6$) ne peuvent apprendre que des coupures linéaires : ils ne voient pT_{τ}^2 que s'ils font deux coupures successives sur pT_{τ} . Cette *feature* rend la dépendance quadratique directement accessible en une seule coupure, réduisant le nombre d'arbres nécessaires.
- **DER_log_tau_pt** : Calculé via $\log(1 + pT_{\tau})$ (logarithme naturel). La distribution de pT_{τ} est très asymétrique avec une longue queue à haute énergie (quelques événements à $pT > 200$ GeV). Le logarithme compresse cette queue et rend la distribution plus gaussienne. Les modèles à arbres peu profonds trouvent des coupures plus pertinentes sur une distribution comprimée que sur une distribution à queue lourde.
- **DER_mass_transverse_tau_met** : Analogie physique de la masse transverse $W \rightarrow \ell\nu$, définie par $\sqrt{2 \cdot pT_{\tau} \cdot MET \cdot (1 - \cos(\Delta\phi(\tau, MET)))}$. Dans $H \rightarrow \tau_{had}\tau_{lep}$, le tau hadronique se désintègre en émettant un neutrino colinéaire. Cette masse transverse est donc corrélée à la masse du tau (~ 1.78 GeV en cinématique) et permet de discriminer le signal contre le fond $W + jets$ où la même quantité correspond au boson W (~ 80 GeV).
- **DER_jet_energy_fraction** : Mesure la fraction de l'énergie visible portée par l'ensemble des jets via le calcul $pT_{jets_all}/(pT_{\tau} + pT_{lep} + pT_{jets_all} + \epsilon)$. Élevée dans les événements $t\bar{t}$ (nombreux jets de haute énergie), elle est faible dans les événements $H \rightarrow \tau\tau$ ggH (souvent 0 jet). Cette variable est complémentaire à **PRI_jet_num** (variable discrète) en apportant une information continue sur l'énergie totale des jets.
- **DER_has_two_jets** : Variable binaire définie par 1 si **PRI_jet_num** ≥ 2 , 0 sinon. Elle isole explicitement la catégorie VBF. Bien que les modèles à arbres puissent apprendre cette coupure sur **PRI_jet_num**, la rendre explicite accélère l'apprentissage : les premières branches de l'arbre peuvent séparer directement les deux régimes cinématiques (VBF et ggH) sans gaspiller de *splits* successifs sur une variable continue.

Bloc	Nb. features	Type	Objectif principal
Kaggle originales (PRI)	17	Mesures brutes	Variables physiques directement mesurées
Kaggle originales (DER)	13	Dérivées Kaggle	Variables pré-calculées par le CERN
Indicateurs _was_missing	8	Binaires	Mémoriser l'absence de jets (crucial pour RN)
Bloc A — DER calculées	16	Physique générale	Masses, distances, ratios, centralités
Bloc B — Jet & Tau pT	10	Physique spécifique	Topologies VBF, régimes cinématiques tau
TOTAL	64	—	Pipeline feature engineering complet

FIGURE 13 – Synthèse du feature engineering

L'ensemble de ce pipeline de 64 features est ensuite traité par l'autoencodeur qui en extrait une représentation latente de 16 dimensions — soit une compression de 75%. Ces 16 dimensions capturent les structures physiques non-linéaires essentielles (régimes de jets, géométrie du système, énergie transverse) dans un espace dense et orthogonal, offrant un point de départ optimal pour les modèles XGBoost et CatBoost.

5.2 Utilité de l'Autoencodeur

Le challenge Kaggle de détection du boson de Higgs pose un problème fondamental : les **250 000 événements d'entraînement** sont chacun décrits par **64 variables** (après *feature engineering*) dont beaucoup sont fortement corrélées entre elles. Des variables comme `PRI_tau_pt`, `DER_energie_totale` et `DER_tau_pt2_over_etot` mesurent essentiellement la même grandeur physique sous des formes différentes.

Un **autoencodeur** est un réseau de neurones non-supervisé qui apprend à **compresser** ces 64 *features* en un vecteur compact — **l'espace latent** (ou code latent, noté z) — puis à les **reconstruire** fidèlement. La contrainte de reconstruction force le réseau à ne garder que l'information **structurellement essentielle** dans le goulot d'étranglement. L'architecture est la suivante :

$$\text{Input}(64) \rightarrow 256 \rightarrow 128 \rightarrow [z : \text{LATENT_DIM}] \rightarrow 128 \rightarrow 256 \rightarrow \text{Output}(64)$$

Chaque bloc de l'encodeur et du décodeur suit le schéma `Linear` $\rightarrow$ `BatchNorm1d` $\rightarrow$ `SiLU`. L'activation **SiLU** (Sigmoid Linear Unit, $x \cdot \sigma(x)$) a été choisie car elle est plus lisse que ReLU sur des données physiques continues, réduisant le risque de neurones morts.

Les avantages clés pour notre problème sont au nombre de quatre. Premièrement, l'apprentissage est **non-supervisé** : le réseau s'entraîne sans utiliser les labels signal/bruit, éliminant tout risque de data leakage. Deuxièmement, les représentations apprises sont **non-linéaires**, capturant des corrélations impossibles à détecter par une PCA classique. Troisièmement, l'erreur de reconstruction par événement fournit un **score d'anomalie** gratuit — un événement physiquement atypique sera mal reconstruit. Quatrièmement, les dimensions latentes sont denses et **orthogonales aux features brutes**, enrichissant l'espace de features sans redondance.

5.3 Problème des valeurs manquantes

5.3.1 La nature physique des -999

Dans le dataset Higgs, les valeurs -999.0 ne sont pas des données manquantes au sens statistique. Ce sont des valeurs **physiquement impossibles encodées intentionnellement** par le CERN pour signaler l'absence d'une quantité mesurable :

- `PRI_jet_leading_pt` = -999 → il n'y a tout simplement pas de jet dans l'événement
- `DER_mass_jet_jet` = -999 → la masse du système jet-jet nécessite ≥ 2 jets ; impossible à calculer avec 0 ou 1 jet
- `DER_deltaeta_jet_jet` = -999 → même raison : séparation angulaire entre deux jets inexistante

Laisser ces valeurs comme NaN dans un réseau de neurones est catastrophique : toute opération matricielle $W \cdot x + b$ propagera le NaN à tous les neurones de la couche suivante. La loss devient NaN, le gradient devient NaN, et les poids ne se mettent jamais à jour.

5.4 La stratégie d'imputation adoptée

La solution retenue combine **deux actions complémentaires** qui présentent l'information physique sans introduire de biais :

Étape 1 — Création d'indicateurs binaires `_was_missing` avant toute modification, pendant que -999.0 est encore présent :

```
df_train['PRI_jet_leading_pt_was_missing'] = (df_train['PRI_jet_leading_pt'] == -999.0).astype(int)
```

Étape 2 — Remplacement de -999.0 par NaN, puis imputation par **zéro**. Le zéro est physiquement cohérent : un jet absent a un pT nul, une masse nulle, une séparation nulle.

Colonne	Signification du -999	Valeur après <code>fillna(0)</code>
<code>PRI_jet_leading_pt</code>	Pas de jet dans l'événement	pT = 0 (jet absent)
<code>PRI_jet_subleading_pt</code>	Pas de 2e jet dans l'événement	pT = 0 (jet absent)
<code>DER_mass_jet_jet</code>	Masse jet-jet incalculable (< 2 jets)	Masse = 0 (système inexistant)
<code>DER_deltaeta_jet_jet</code>	Séparation η incalculable (< 2 jets)	$\Delta\eta$ = 0 (jets confondus)
<code>PRI_jet_leading_eta</code>	Pas de jet leading	η = 0 (central par défaut)
<code>PRI_jet_leading_phi</code>	Pas de jet leading	ϕ = 0
<code>PRI_jet_subleading_eta</code>	Pas de 2e jet	η = 0
<code>PRI_jet_subleading_phi</code>	Pas de 2e jet	ϕ = 0

FIGURE 14 – Stratégie imputation

Après l'application de la stratégie, il ne reste plus de NaN. Les 8 indicateurs `_was_missing` jouent un rôle crucial pour le réseau de neurones : ils lui permettent d'apprendre séparément la règle "cet événement a un jet" de la règle "si le jet existe, quelle est sa valeur de p_T ". Sans ces indicateurs, un zéro obtenu par imputation est indiscernable d'un zéro "réel".

5.5 Recherche de la dimension optimale du goulot d'étranglement

5.5.1 Principe de la méthode du coude

Pour déterminer la taille optimale de l'espace latent, on entraîne un autoencodeur léger (30 epochs) pour chaque valeur candidate de `LATENT_DIM` $\in \{4, 8, 12, 16, 20, 24, 32, 48\}$, et on mesure l'erreur de reconstruction MSE sur la validation.

La MSE (Mean Squared Error) mesure la fidélité de la reconstruction :

$$\text{MSE} = \frac{1}{n \cdot d} \times \sum_i \sum_j (x_{ij} - \hat{x}_{ij})^2$$

où x_{ij} est la feature j de l'événement i et $\hat{x}_{ij}$ sa valeur reconstruite par l'autoencodeur. Une MSE faible signifie une bonne reconstruction.

La dimension optimale est le **coude de la courbe MSE** — le point au-delà duquel ajouter des dimensions supplémentaires n'apporte plus de gain significatif en reconstruction. Ce point est détecté automatiquement par la méthode de la distance perpendiculaire maximale à la droite reliant le premier et le dernier point.

5.5.2 Résultats – Graphiques de sélection

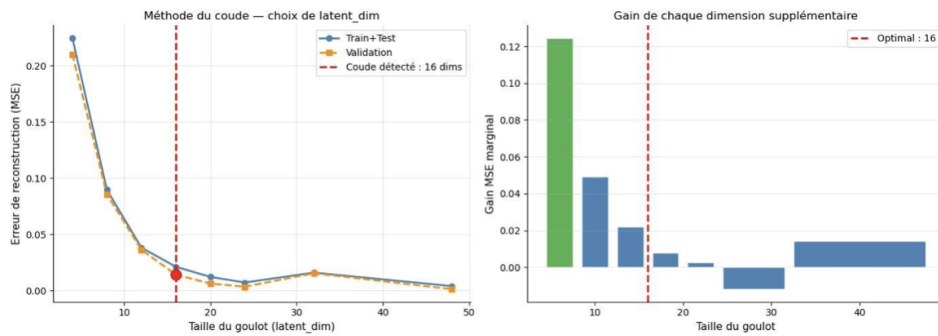


FIGURE 15 – Méthode du coude (gauche) et gain marginal par dimension (droite). Le coude est détecté à `LATENT_DIM` = 16.

Le graphique de gauche montre la courbe MSE en fonction de la taille du goulot, pour le dataset d'entraînement (bleu) et de validation (orange pointillé). Les deux courbes sont très proches tout au long, absence d'overfitting confirmée. Le coude est clairement visible à 16 dimensions : la MSE passe de 0.22 (4 dims) à 0.015 (16 dims), puis stagne autour de 0.005.

Le graphique de droite (gain marginal) confirme : la barre verte (passage de 4 à 8 dims) apporte le gain le plus important (0.12 MSE). À partir de 16 dims, les gains deviennent marginaux.

voire négatifs — signe que les dimensions supplémentaires encodent du bruit plutôt que de la structure.

5.5.3 Convergence de l'entraînement final

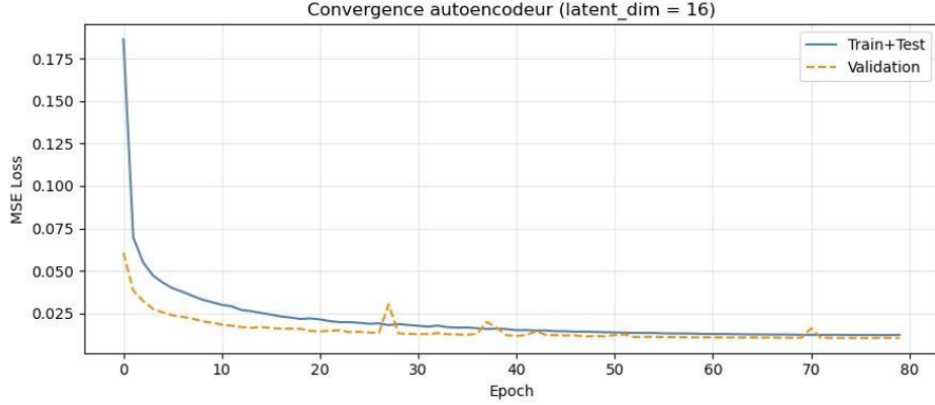


FIGURE 16 – Courbes de convergence de l'autoencodeur final (LATENT_DIM = 16, 80 epochs). Train en bleu, Validation en orange pointillé.

La courbe de convergence montre un comportement sain : la loss train (bleue) et la loss validation (orange) convergent ensemble vers 0.015 en 80 epochs. Les deux petits pics visibles autour des epochs 27 et 37 sont caractéristiques du scheduler cosinus — des remontées contrôlées de learning rate qui aident à sortir de minima locaux. MSE finale ≈ 0.014 , ce qui confirme une très bonne reconstruction.

5.5.4 Analyse de l'espace latent — PCA 2D

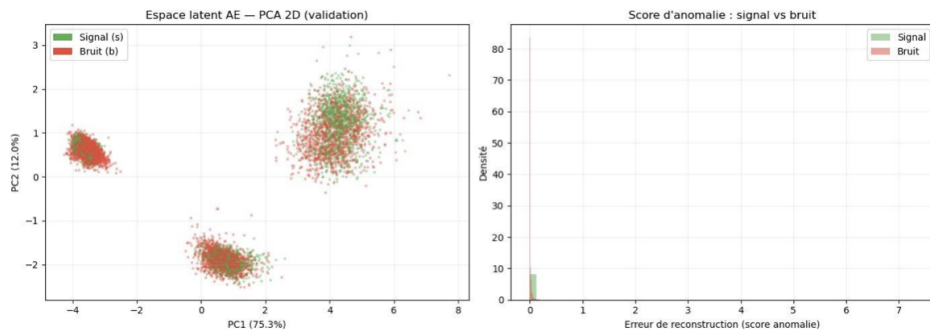


FIGURE 17 – Gauche : espace latent projeté en PCA 2D (signal en vert, bruit en rouge). Droite : distribution du score d'anomalie par classe.

La projection PCA 2D de l'espace latent révèle 3 clusters distincts, correspondant avec une forte probabilité aux trois catégories de jets du dataset ($PRI_jet_num = 0, = 1, \geq 2$). L'autoencodeur a appris sans supervision que le nombre de jets est la structure dominante des données physiques — une validation remarquable de son utilité.

PC1 explique 75.3% de la variance et PC2 explique 12.0%, soit 87.3% au total. Signal et bruit sont mélangés dans chaque cluster, ce qui est attendu : l'autoencodeur optimise la reconstruction,

pas la discrimination. La séparation signal/bruit est déléguée aux modèles supervisés (XGBoost, CatBoost).

Le score d'anomalie (graphique droite) est concentré près de zéro pour les deux classes — confirmant la MSE de 0.014 : presque tous les événements sont bien reconstruits. Les quelques outliers à score élevé (jusqu'à 3.5) correspondent à des événements cinématiquement atypiques qui seront potentiellement utiles comme signal discriminant.

6 Conclusion

Ce projet nous a permis de nous confronter à une problématique réelle et complexe de la physique des hautes énergies : la détection du boson de Higgs. Pour répondre aux exigences de ce challenge, nous avons identifié et mis en œuvre des méthodes de Machine, notamment les algorithmes de Gradient Boosting tels que XGBoost, CatBoost et LightGBM.

La réussite de notre approche repose sur une synergie entre les différentes parties du projet. D'une part, un **feature engineering** rigoureux a permis de transformer les mesures brutes du détecteur en de nouvelles variables physiques discriminantes, facilitant ainsi l'apprentissage de structures complexes par nos modèles.

L'optimisation bayésienne via Optuna, couplée à une analyse de l'espace des hyperparamètres par l'algorithme **UMAP**, nous a permis de garantir la robustesse de certaines solutions et de confirmer la convergence vers des optima globaux stables. Cette rigueur méthodologique a été validée par notre score final de **3,695** sur le *Private Leaderboard* de Kaggle avec XGBoost. Ce résultat, cohérent avec nos performances en entraînement, démontre une excellente capacité de généralisation et une absence de surapprentissage, validant ainsi la pertinence de nos choix techniques.

En conclusion, ce projet illustre le potentiel des méthodes d'ensemble et de l'apprentissage non-supervisé pour la découverte scientifique. Bien que nos modèles actuels atteignent des performances remarquables, l'intégration de techniques de *stacking*, l'utilisation de réseaux de neurones profonds sur des volumes de données encore plus vastes, et la décomposition suivant le nombre de jets constituent des perspectives prometteuses pour affiner davantage la précision des mesures de significativité statistique pour ce challenge.

Références

- [1] Kaggle, *Higgs Boson Machine Learning Challenge*,
<https://www.kaggle.com/competitions/higgs-boson>
- [2] XGBoost Developers, *XGBoost Documentation*,
<https://xgboost.readthedocs.io/>
- [3] Microsoft, *LightGBM Documentation*,
<https://lightgbm.readthedocs.io/>
- [4] Yandex, *CatBoost Documentation*,
<https://catboost.ai/docs/>
- [5] Leland McInnes, John Healy, and James Melville, *UMAP : Uniform Manifold Approximation and Projection for Dimension Reduction*,
<https://umap-learn.readthedocs.io/>
- [6] Optuna, *A hyperparameter optimization framework*,
<https://optuna.org/>